



GEBZE TECHNICAL UNIVERSITY
Department of Computer Engineering
CSE 396 — Computer Engineering Project | Spring 2026

Project Proposal

AI-Powered Smart PPE Inspection Station

GROUP-11



Group-11 Members

Student Name	Student Number
Muhammed Emir Kara	210104004071
Mümincan Durak	210104004057
Hüseyin Elyesa Yeşilyurt	210104004080
Alperen Söylen	220104004024
Ahmet Emre Kurt	220104004016
Zeynep Etik	220104004035
Emre İlhan Şenel	230104004907
Tarık Saeede	200104004804

1. Project Definition and Aims

1.1 Project Summary

The AI-Powered Smart PPE Inspection Station is an automated safety gate system designed for industrial environments such as construction sites and factories. The system integrates RFID-based identity verification with a Computer Vision AI model running on a Raspberry Pi 5 to ensure that every worker entering a hazardous zone is equipped with the mandatory Personal Protective Equipment (PPE) required for their specific role.

The station operates as a two-sided architecture: an Admin Side where personnel are registered and roles are assigned, and a Turnstile Side where real-time inspection and access control takes place. A central backend server connects both sides over a local network.

1.2 Problem Definition

Traditional safety inspections at industrial entry points rely on manual checks performed by personnel, which are prone to human error, inconsistency, and are not scalable for high-traffic zones. There is currently no automated, role-aware system that can verify both the identity of a worker and their compliance with role-specific PPE requirements simultaneously, without human intervention.

This project addresses that gap by combining RFID-based identification with edge AI inference, creating a system that is both fully automated and context-aware — adapting required PPE checks based on the worker's assigned role stored in a central database.

1.3 Project Aims

- **Automated Safety Compliance:** Eliminate human error in PPE inspection at industrial entry points.
- **Role-Based Access Control:** Determine required equipment based on the worker's job role retrieved from the backend database.
- **Real-Time AI Inspection:** Run a computer vision model on Raspberry Pi 5 to detect PPE items (helmet, vest, gloves, etc.) in real time.
- **Worker Registration & Management:** Provide an admin panel for registering workers, linking RFID cards, and assigning job roles.
- **User Feedback:** Display personalized, real-time feedback on a screen (e.g., missing equipment list, welcome messages) at the turnstile.
- **Data Logging & Reporting:** Record every entry attempt with timestamp, identity, detected equipment, and access result for administrative audits.
- **Physical Access Control:** Control a servo-driven turnstile gate to physically allow or deny entry based on inspection results.

1.4 Scope and Limitations

In Scope:

- Indoor, controlled-lighting industrial environment.
- RFID card-based identity verification (RC522 module).
- Role-based PPE requirement lookup from a centralized server database.

- Real-time PPE detection via AI model on Raspberry Pi 5 with AI Kit.
- A display for turnstile-side to inform the user.
- Admin panel application (web or desktop) for worker registration and role assignment.
- Communication between Raspberry Pi and central server on the same local network.
- Servo motor-controlled physical turnstile gate.

Out of Scope:

- Outdoor environments or GPS-based tracking.
- Facial recognition or biometric identification (RFID only).
- Multi-zone or multi-turnstile coordination.
- Cloud-based deployment or remote access over the internet.
- Detection of PPE worn incorrectly (only presence/absence is checked).

2. Literature Review

2.1 PPE Detection Using Computer Vision

The automated detection of Personal Protective Equipment in industrial and construction environments has been an active area of research in recent years. Early approaches relied on traditional image processing techniques such as color-based segmentation and histogram analysis to identify safety helmets and vests. However, these methods proved fragile under varying lighting conditions and complex backgrounds typical of real industrial sites.

The introduction of deep learning-based object detection architectures significantly improved detection performance. Convolutional Neural Network (CNN) based detectors, particularly the YOLO (You Only Look Once) family of models, became the dominant approach for PPE detection due to their balance of speed and accuracy suitable for real-time applications. Studies such as Nath et al. (2020) demonstrated that YOLO-based models could achieve high detection accuracy for safety helmets, vests, and gloves in construction site imagery. More recent work has explored lightweight model variants specifically designed for deployment on resource-constrained hardware, showing that acceptable accuracy can be maintained with significantly reduced computational requirements through techniques such as model pruning and quantization.

Public datasets specifically targeting PPE detection have also emerged, including construction site datasets that cover the key equipment classes relevant to this project: hard hats, safety vests, gloves, safety boots, and face masks. These datasets provide a foundation for training and fine-tuning models without requiring entirely new data collection from scratch.

2.2 Edge AI Inference on Embedded Systems

Deploying AI inference on edge devices such as single-board computers presents challenges related to computational power, memory constraints, and thermal management. Raspberry Pi platforms have been widely studied as edge inference targets due to their low cost and accessibility. Research has shown that standard deep learning models trained for desktop or cloud environments are too large and computationally expensive to run efficiently on such hardware without optimization.

Model optimization techniques have been shown to reduce inference latency by a factor of three to ten times compared to unoptimized models, with only marginal losses in detection accuracy. The introduction of the Raspberry Pi AI Kit, which integrates a dedicated NPU accelerator, further extends the feasibility of real-time object detection on the platform. Studies benchmarking lightweight detection models on Raspberry Pi hardware report inference times in the range of one to three seconds per frame on optimized models, which is within the acceptable range for an access control inspection scenario.

2.3 RFID-Based Access Control in Industrial Environments

Radio Frequency Identification (RFID) technology has been widely adopted in industrial access control systems for worker identification and zone management. ISO 14443A-compliant RFID cards and readers, such as the RC522 module, provide a cost-effective and reliable method for

contactless identity verification at entry points. Existing industrial access control systems typically use RFID solely for identity verification without any additional safety compliance checking.

Research on smart access control systems highlights the limitation that current RFID-only systems cannot enforce safety compliance requirements. They can prevent unauthorized personnel from entering restricted areas but cannot detect whether authorized workers are wearing the equipment required for their specific role or the specific hazard zone they are entering. This represents a significant gap in automated industrial safety management.

2.4 Existing Smart Safety Gate Solutions

Several commercial and academic systems have attempted to address automated PPE compliance at entry points. Commercial solutions such as SmartCone and Brainhub Safety Systems offer camera-based PPE detection at construction site entrances. However, these systems are typically cloud-dependent, require high-end hardware, and do not integrate identity verification with role-specific PPE requirements. They detect whether any person in frame is wearing a helmet or vest, but cannot determine whether that specific person's role requires additional equipment such as gloves or face masks.

3. Methodology

3.1 Overall Approach

This project follows an edge AI and IoT integration approach, combining real-time computer vision inference with hardware-level sensor integration and a centralized data management server. The system is designed as a two-sided architecture. All components communicate over a local network, eliminating any dependency on cloud infrastructure and ensuring the system remains operational in environments with restricted internet access.

The development follows a modular approach where each functional area of the system is developed as an independent module with a clearly defined interface. Modules are developed in parallel and integrated progressively throughout the project timeline, following Agile development standards with weekly synchronization meetings and API contracts defined before development begins.

3.2 Data Strategy

The object detection model requires a labeled image dataset of workers wearing various combinations of PPE items in industrial environments. The primary data source for this project is the Ultralytics Construction PPE Dataset, which contains annotated images covering the target detection classes: hard hat, safety vest, gloves, safety boots, and face mask. This dataset provides a foundation of real-world construction site imagery with varied backgrounds, lighting conditions, and worker poses.

Dataset Structure

The Construction-PPE dataset is organized into three main subsets:

- **Training Set:** The primary collection of annotated construction images featuring workers with both complete and partial PPE usage.
- **Validation Set:** A designated subset used to fine-tune and assess model performance during PPE detection and compliance monitoring.
- **Test Set:** An independent subset reserved for evaluating the final model's effectiveness in detecting PPE and identifying compliance issues.

To improve model robustness and generalization, the dataset will be augmented using geometric transformations such as flipping, rotation, and cropping to simulate different worker orientations and camera angles, as well as photometric adjustments including brightness and contrast variations to simulate the indoor lighting conditions of the target deployment environment.

3.3 Techniques

Object Detection Model: A lightweight single-stage object detection architecture, such as a YOLO family variant, will be used for PPE detection, selected for its balance of detection accuracy and inference speed on resource-constrained hardware. The model will be fine-tuned on the PPE dataset using transfer learning from a pre-trained base checkpoint to reduce training time and improve accuracy.

Model Optimization: Following training, the model will be converted to ONNX (Open Neural Network Exchange) format for hardware-agnostic deployment, and further optimized using INT8 post-training quantization. INT8 quantization reduces model size and inference latency by approximately 3-4x compared to full precision models with marginal accuracy loss. The Raspberry Pi AI Kit's dedicated NPU (Neural Processing Unit) will be utilized for accelerated inference, targeting an inspection cycle time of under 10 seconds from card scan to gate decision.

Identity Verification: Worker identity verification is performed using RC522 RFID readers operating on the ISO14443A protocol. Each worker is assigned a unique RFID card. The card ID is read via SPI communication on the Raspberry Pi and sent to the backend server for identity lookup. The same protocol is used on both the Admin Side for card registration and the Turnstile Side for worker identification.

Backend and Data Management: The central backend server exposes a REST API over HTTP, following standard RESTful design principles. All persistent data including worker profiles, role definitions, PPE requirement mappings, and entry log records is stored in a relational database. The access decision logic compares the detected PPE list returned by the AI module against the required PPE list retrieved from the database for the worker's assigned role, producing a binary PASS or FAIL result with a list of missing items when applicable.

Communication Protocol: All communication between system components including the Admin Panel to server, Raspberry Pi to server, and tablet display to server uses HTTP REST calls over the local network. For real-time display updates on the tablet, a WebSocket connection will be used to push inspection results from the server to the tablet application without polling.

Physical Access Control: Gate access is controlled by high-torque servo motors connected to the Raspberry Pi GPIO pins via PWM signals. The gate controller module exposes a simple open/close API consumed by the IoT module upon receiving an access decision from the backend.

3.4 Development Approach

The project follows Agile development standards adapted for a student team of eight members working across five modules. The backend server can be run locally on any team member's machine during development and later deployed on a dedicated machine. The key practices applied are:

API-First Development: All module interfaces and REST API contracts are defined and agreed upon in Phase 1 before any module begins implementation. This allows all five modules to be developed in parallel without blocking dependencies.

Feature-Branch Git Workflow: Each module has its own development branch in the shared GitHub repository. Code is merged into the main branch only after peer review, ensuring integration stability throughout the project.

Mock APIs During Development: Modules that depend on other modules use mock API responses during early development so that work can proceed before the dependent module is complete.

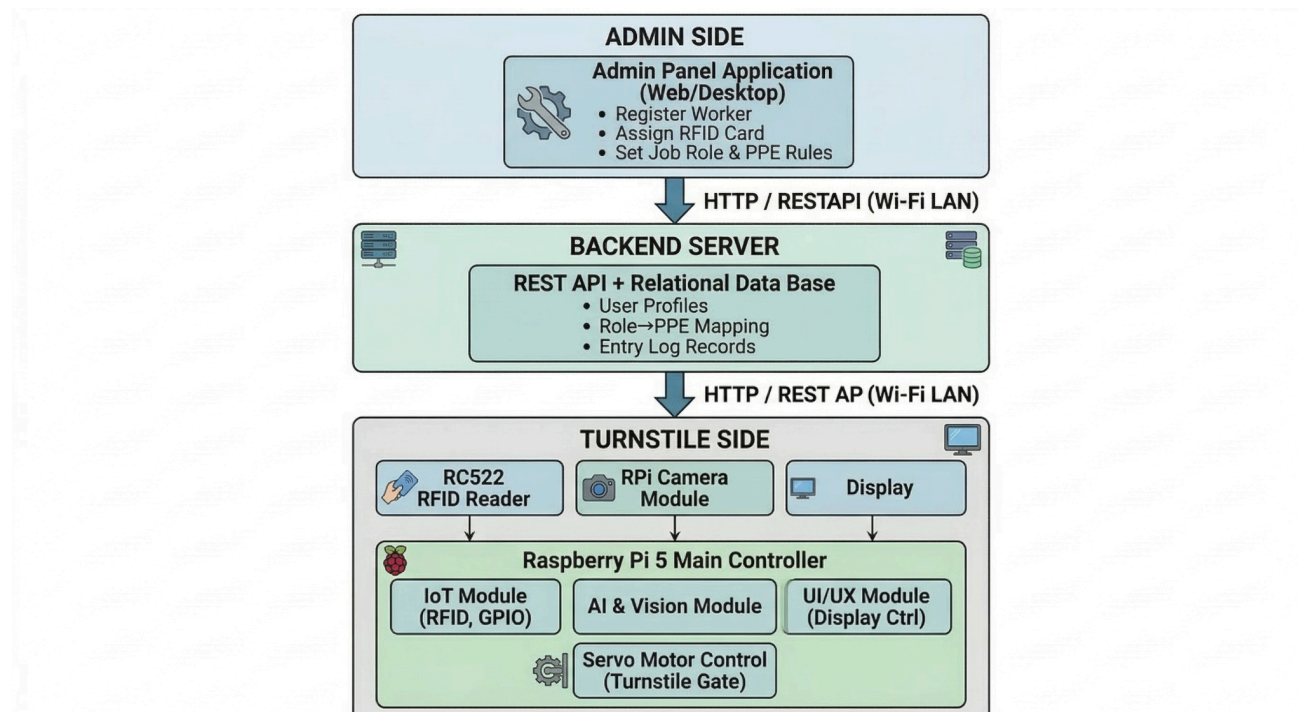
Weekly Stand-Up Meetings: The team holds weekly synchronization meetings to track progress, identify blockers, and synchronize module interfaces. Integration issues are caught early rather than discovered only in the final integration phase.

Testing Strategy: Each module is individually unit-tested before integration. End-to-end tests are performed on the physical system during the integration phase, covering the three core scenarios: a compliant worker with full PPE, a worker with missing PPE, and an unregistered card.

4. Technical Architecture

4.1 System Architecture Overview

The system is composed of two physical units connected to a central backend server over a local network.



4.2 Software Architecture

The software stack is organized into four layers:

Data Layer:

A Relational Database hosted on the backend server stores all persistent data: user profiles, role definitions, PPE requirement mappings, and entry log records.

Backend Layer:

A REST API server exposes endpoints consumed by both the Admin Panel and the Raspberry Pi. It handles worker registration, role/PPE lookups, and log writes.

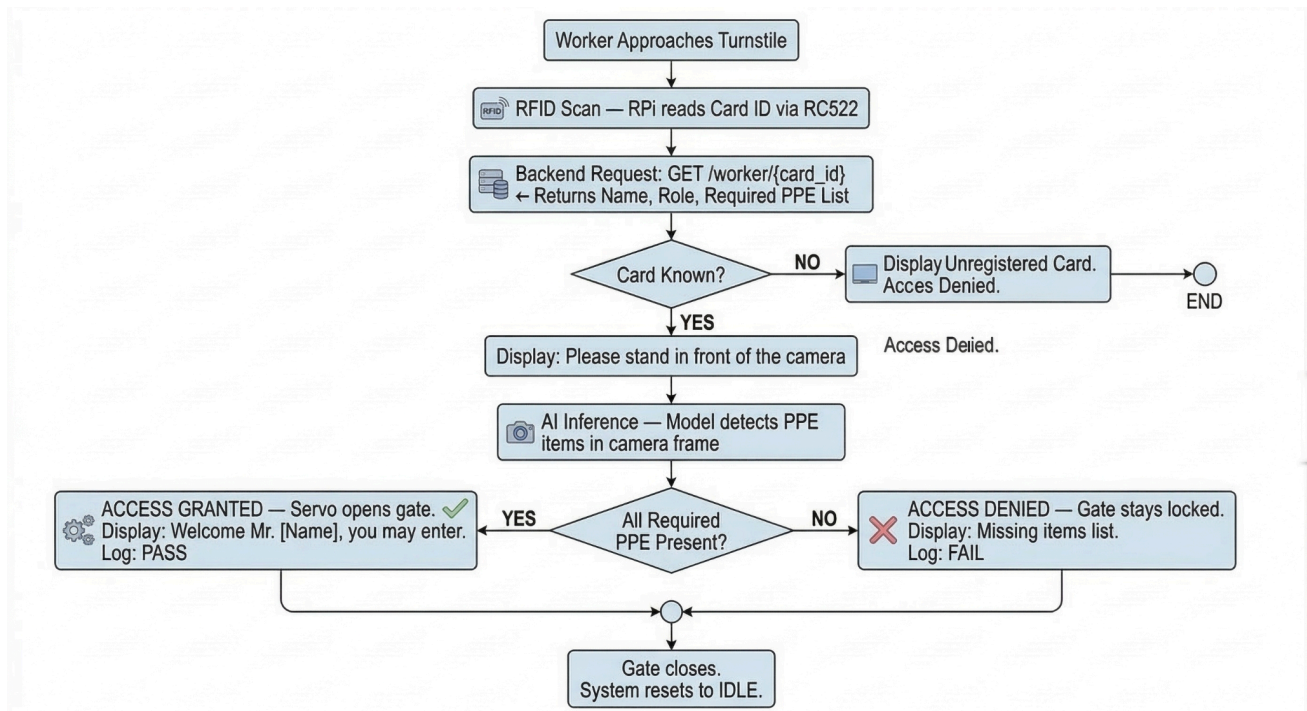
Application Layer:

The Admin Panel (web or desktop application) communicates with the backend via HTTP REST calls. The Raspberry Pi runs its local modules (IoT, AI Vision, UI/UX) and communicates with the backend over a local network.

Hardware Abstraction Layer:

On the Raspberry Pi, GPIO drivers manage the RC522 RFID reader, servo motors, and the display. Camera frames are captured and preprocessed by the vision pipeline and fed to the model inference engine.

4.3 System Flow Diagram



4.4 AI Model Strategy

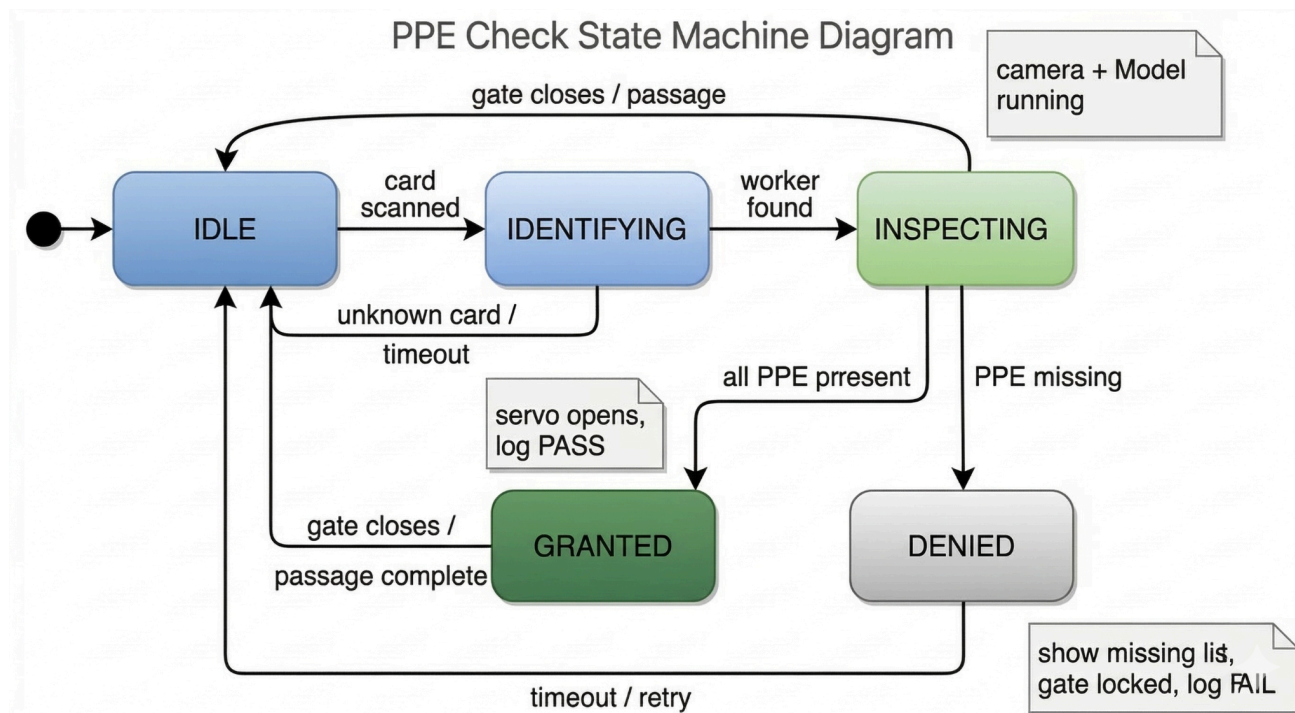
The AI inspection pipeline uses a lightweight object detection model fine-tuned on a PPE dataset. The model detects the following classes: hard hat, safety vest, gloves, safety boots, and face mask. It is optimized for efficient edge inference on the Raspberry Pi 5 with AI Kit.

Inference Prompt Logic:

```

Required PPE List (from backend): ["hard_hat", "safety_vest", "gloves"]
Model Detections:                  ["hard_hat", "safety_vest"]
Missing items:                     ["gloves"]
Decision:                          ACCESS DENIED — show missing items on display
  
```

4.5 State Machine



5. Hardware Requirements

5.1 Component List

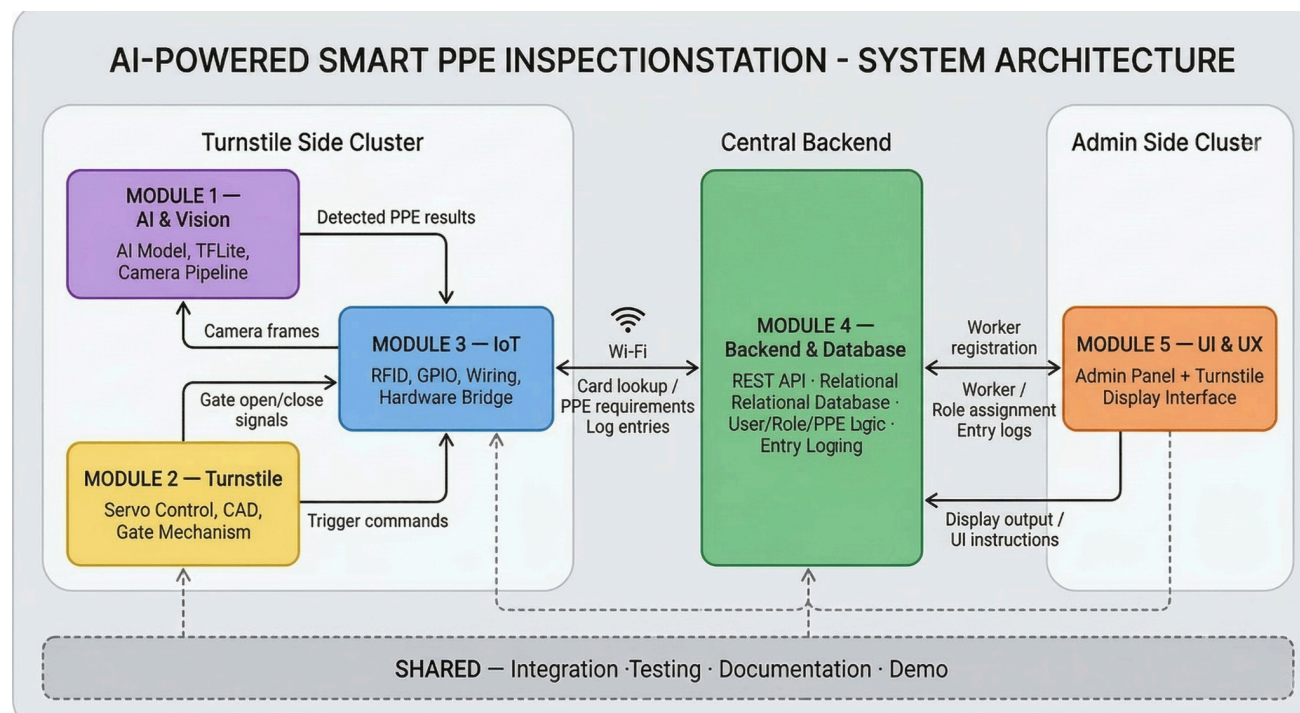
#	Component	Model / Description	Qty	Unit (TL)	Total (TL)
1	Raspberry Pi 5	8GB RAM + AI Kit	1	4.826,12	4.826,12
2	RPi Camera Module	Camera Module V3 (12MP)	1	2.034,28	2.034,28
3	RFID Reader (Turnstile)	RC522 RFID NFC Module	2	61,23	122,46
4	RFID Cards / Tags	ISO14443A Cards	10	7,18	70,18
5	Servo Motors	High-torque servo motor (gate mechanism)	2	188,00	376,00
6	Display / Screen	A Tablet or HDMI Monitor	1	—	—
7	Power Supply	Lipo Battery	1	1.530,42	1.530,42
8	Chassis / Structure	3D-printed or acrylic frame for turnstile	1	1385,92	1385,92
9	MicroSD Card	Samsung Evo Plus 128 GB Micro SD Kart	1	1.199,00	1.199,00
10	Wiring & Breadboard	Jumper cables, GPIO breakout	1 set	132,13	132,13
				TOTAL	11.676,51

5.2 Software Requirements

Software	Purpose	License
Raspberry Pi OS (64-bit)	Operating system for RPi 5	Free
Python 3.11+	Primary development language	Open Source
React / HTML+JS (or PyQt)	Admin panel frontend	Open Source
ONNX Runtime	Model inference optimization on RPi	Open Source
OpenCV	Camera capture and image preprocessing	Open Source
gpiozero / RPi.GPIO	GPIO control for RFID, servo, display	Open Source
Git / GitHub	Version control	Free (student)

6. Module Structure and Task Distribution

6.1 Module Overview



6.2 Module Details

MODULE 1: AI & Vision Module (2 people — Zeynep & Alperen)

Responsibility: Training, optimizing, and deploying the PPE detection model on the Raspberry Pi 5 with AI Kit.

- Collecting and preparing a PPE image dataset primarily from the Ultralytics Construction PPE Dataset, covering hard hats, safety vests, gloves, safety boots, face masks, and safety goggles. Roboflow Universe and Kaggle will be used as supplementary sources to increase sample diversity where needed. Augmentation (brightness variation, rotation, zoom, noise) will be applied to improve model robustness under indoor lighting conditions.
- Training a lightweight object detection model on the labeled dataset.
- Converting the trained model into an optimized format for edge deployment on the Raspberry Pi 5.
- Integrating the model with the camera pipeline (frame capture → inference).
- Benchmarking inference speed and accuracy on the Raspberry Pi 5 hardware.
- Providing a Python inference API consumed by the Backend module: `detect_ppe(frame) → list of detected items`.

Outputs:

- Model inference module (load, predict, optimize)
- Frame capture and preprocessing pipeline
- Performance benchmarking tool
- Model performance report (inference time, accuracy, memory usage)
- Dataset composition report (class distribution, source breakdown, augmentation strategy)

MODULE 2: Turnstile Module (2 people — Mümincan & Emir)

Responsibility: Physical design, fabrication, and servo control of the turnstile gate mechanism.

- Designing the turnstile arm and rotation mechanism using CAD software.
- Fabricating the physical gate structure using 3D printing or acrylic/aluminum materials.
- Implementing servo motor control logic: gate unlock, rotate, and re-lock based on system signals.
- Integrating servo control with the Raspberry Pi GPIO.
- Providing a gate control API: `gate.open()` and `gate.close()` for use by the IoT module.

Outputs:

- Gate controller module (servo motor control)
- CAD design files
- Physical turnstile assembly

MODULE 3: IoT Module (Support role — Mümincan, Emir, Alperen)

Responsibility: Hardware-software bridge, RFID integration, GPIO layout, and wiring of all components on the Raspberry Pi.

- Developing the RFID reader interface for both the turnstile and admin sides.
- Managing GPIO pin assignments for camera, RFID, servo motors, and display.
- Wiring design and power distribution for all connected peripherals.
- Setting up and integrating the tablet-based display interface into the turnstile system.
- Ensuring stable communication between all hardware components and the Raspberry Pi main controller.
- Providing an RFID reading API: `rfid.read_card()` → `card_id`.

Outputs:

- RFID reader interface module
- GPIO pin configuration and management module
- Hardware wiring schematic

MODULE 4: Backend & Database Module (2 people — Ahmet & Elyesa)

Responsibility: Central server, data logic, REST API, and database management.

- Designing the relational database schema: Users table (ID, name, RFID card ID, role), Roles table (role name, required PPE list), Entry Log table (timestamp, card ID, detected PPE, result, missing items).
- Implementing a REST API server with endpoints for worker registration, RFID card lookup, PPE requirement retrieval, and entry log writes.
- Implementing the access decision logic: compare detected PPE list from AI module against required PPE list from database.
- Building the admin-side RFID registration flow: scan new card → link to worker profile → assign role.
- Ensuring the API is accessible over the local network from both the Admin Panel and the Raspberry Pi.

Outputs:

- REST API server module
- Database manager module
- Data models (User, Role, Log)
- API documentation (endpoint list, request/response formats)

MODULE 5: UI & UX Module (2 people — Tarik & Emre)

Responsibility: User-facing interfaces for both the Admin Panel (PC side) and the Turnstile Display (RPI side).

- Admin Panel: Worker registration form, RFID card assignment, job role selection, and worker list management.
- Admin Panel: Entry log viewer and compliance statistics dashboard.
- Turnstile Tablet Display: Real-time camera feed with bounding boxes around detected PPE items.
- Turnstile Tablet Display: Animated instructions.
- Turnstile Tablet Display: Access Granted screen.
- Turnstile Tablet Display: Access Denied screen.

Outputs:

- Admin Panel application (communicates with backend REST API)
- Turnstile tablet display application
- UI/UX design files and style guide

SHARED RESPONSIBILITIES (All Members)

- System integration: connecting all modules end-to-end.
- End-to-end scenario testing (registered worker with full PPE, missing items, unregistered card).
- Performance metric measurement: detection accuracy, inspection cycle time, false positive/negative rates.
- Git repository management and code review.
- Technical documentation and user guide.
- Final demo preparation and presentation.
- Demo video recording.

6.3 Person–Module Assignment Matrix

Person	Module 1 (AI & Vision)	Module 2 (Turnstile)	Module 3 (IoT)	Module 4 (Backend)	Module 5 (UI & UX)
Zeynep	P		S		
Alperen	S		P		
Mümincan		P	S		

Person	Module 1 (AI & Vision)	Module 2 (Turnstile)	Module 3 (IoT)	Module 4 (Backend)	Module 5 (UI & UX)
Emir	S	P			
Ahmet		S		P	
Elyesa			S	P	
Tarık				S	P
Emre			S		P

P = Primary (main responsibility) S = Secondary (support & cross-learning)

7. Project Timeline

7.1 Weekly Gantt Chart (12 Weeks)

	PHASE 1 — Setup & Planning		PHASE 2 — Active Development						PHASE 3 — Integration & Testing			PHASE 4 — Demo	
	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7	Week 8	Week 9	Week 10	Week 11	Week 12	
Row 1 — Module 1: AI & Vision	Research		Development of Model								Testing and Fine-Tuning		◆ Final Demo
		◆ RPi boots, camera works, RFID scans		◆ Model detects first PPE class on RPi									
Row 2 — Module 2: Turnstile	CAD Design		Fabrication + Servo Control						Integrate & Test				
				◆ Backend API operational		◆ Turnstile gate opens/closes physically		◆ All modules individually		◆ Full inspection cycle working			
Row 3 — Module 3: IoT	Hardware Planning & Wiring Design	RFID Module Design	RPi Setup + GPIO + Camera Pipeline		Display Module Design & Development		Module Integration into Turnstile		Hardware Support, Debugging & Testing				
				◆ Backend API operational		◆ RFID → Backend → PPE flow works end-to-end							
Row 4 — Module 4: Backend & Database	Database Schema Design & API Contract Definitions	Database Implementation & Server Setup	Worker Registration & Role-PPE Requirement Logic Engine		Access Decision Logic & Entry Logging System		Integration with Module 5 (UI & UX) and Module 3 (IoT)		API Testing, Bug Fixes & Stabilization				
Row 5 — Module 5: UI & UX	Wireframes & UI/UX Design Planning	Admin Panel Foundation & Backend Connection	Worker Registration, Role Assignment & Card Management Screens		Entry Log Viewer & Compliance Statistics Dashboard		Integration with Module 4 (Backend) and Module 3 (IoT)		UI Polish, Testing & Bug Fixes				

Dark fill = Preparation / Research

Medium fill = Active Development

Light fill = Refinement / Support

◆

= Milestone

7.2 Phase Details

PHASE 1 — Setup & Planning (Weeks 1–2)

- Hardware procurement and Raspberry Pi 5 environment setup.
- Development environment setup for all team members (Python, Git, IDE).
- Database schema design and API contract definitions between modules.
- CAD design initiation for turnstile chassis.
- Object detection model research and initial dataset collection strategy
- Milestone: RPi boots, camera captures frames, RFID reader scans a card.

PHASE 2 — Active Development (Weeks 3–9)

- All modules developed in parallel with weekly integration sync meetings.
- Week 4 Milestone: Object detection model running on RPi and detecting at least one PPE class
- Week 5 Milestone: Backend API operational — worker registration and card lookup working.
- Week 6 Milestone: Turnstile physical structure fabricated; servo open/close functional.
- Week 7 Milestone: RFID → backend query → PPE requirement retrieval flow working end-to-end.
- Week 9 Milestone: All individual modules complete and unit-tested.

PHASE 3 — Integration & Testing (Weeks 9–11)

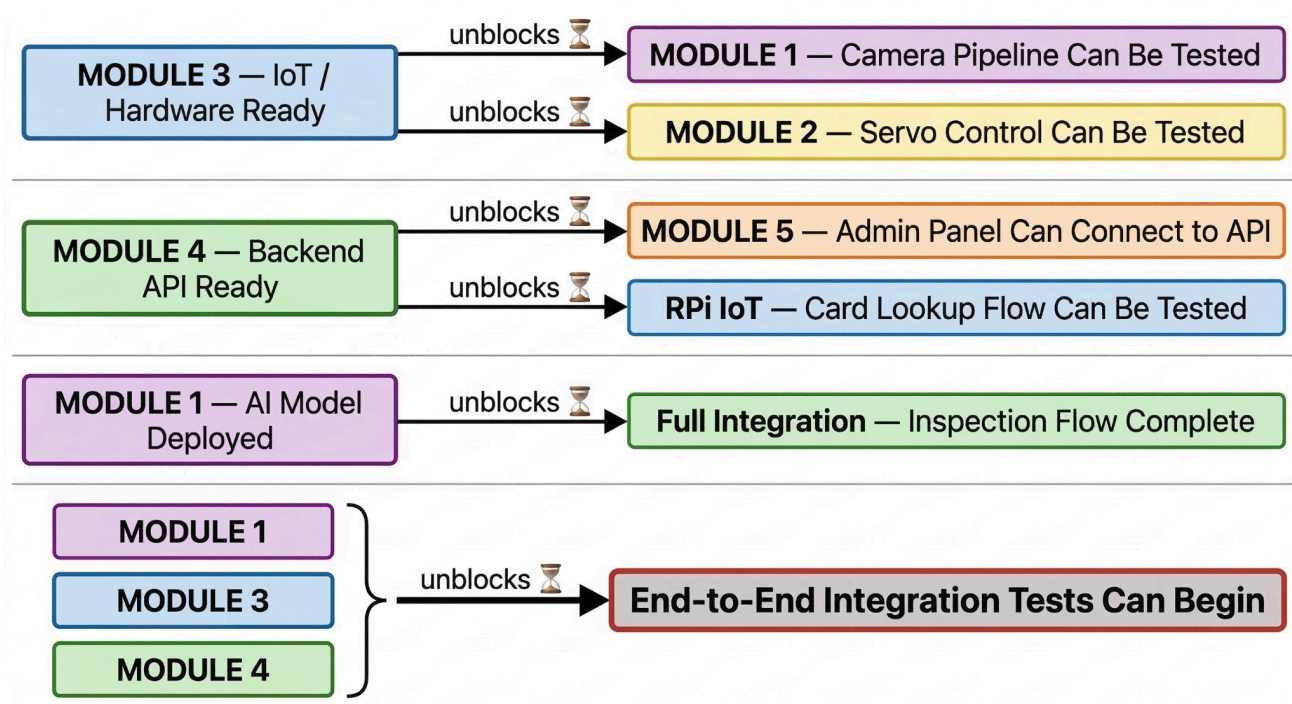
- Full system integration: turnstile flow from card scan to gate open/close.
- End-to-end scenario tests: compliant worker, missing PPE, unregistered card.
- Performance measurement: detection accuracy, inspection cycle time.

- Bug fixing and performance optimization.
- Week 10 Milestone: Complete inspection cycle works for basic scenario.

PHASE 4 — Demo & Documentation (Week 12)

- Final demo preparation and rehearsal.
- Project report and technical documentation completion.
- Demo video recording.
- Presentation preparation.

7.3 Critical Dependencies



8. Cost Analysis

8.1 Hardware Costs

Category	Items	Cost (TL)
Main Processor	Raspberry Pi 5 (8GB) + AI Kit + MicroSD	4.826,12
Vision	RPi Camera Module V3	2.034,28
Identity (RFID)	2x RC522 Module + ISO14443A Cards (x10)	172,64
Actuation	High-torque Servo Motors (x2)	376,00
Display	7" Touchscreen / HDMI Monitor	—
Structure	3D-printed / acrylic chassis + fasteners	1385,92
Power	5V/5A power supply + cables	1.530,42
Misc.	Jumper cables, breadboard, GPIO breakout	1331,13
	HARDWARE TOTAL	11.676,51

8.2 Software Costs

All software components used in this project are open-source or free. No additional licensing costs are incurred.

Software	License	Cost
Raspberry Pi OS, Python, OpenCV	Open Source	0
ONNX Runtime	Open Source	0
React / HTML+JS or PyQt	Open Source	0
Git / GitHub	Free (student)	0

8.3 Total Project Budget

Category	Cost (TL)
Hardware (components, chassis, display)	11.676,51
Software	0
TOTAL	11.676,51

9. Risk Analysis and Mitigation

Risk	Likelihood	Impact	Mitigation
Model inference too slow on RPi (>5s/frame)	Medium	High	Use INT8 quantization and the AI Kit NPU; reduce input resolution; fallback to a lighter model variant.
Low PPE detection accuracy in varied lighting	Medium	High	Collect diverse training data; add data augmentation; tune confidence threshold.
RFID reader hardware failure	Low	Medium	Keep a spare RC522 module; test both units early in Phase 1.
Servo motor insufficient torque for gate	Low	Medium	Select servo with torque rating verified against gate design; test in Phase 2.
Backend API unreachable from RPi over Wi-Fi	Low	High	Test network stability early; implement retry logic on RPi; use local fallback cache.
Module integration delays	High	High	Define API contracts in Week 1; weekly integration sync meetings; use mock APIs during development.
Dataset insufficient for model accuracy	Medium	High	Use public PPE datasets as a supplement; apply transfer learning.
Team communication or coordination issues	Medium	Medium	Weekly stand-up meetings; transparent task tracking on GitHub Projects.

10. Expected Outputs and Success Criteria

10.1 Minimum Success Criteria (Must Have)

- The RFID reader correctly identifies a registered worker by card scan.
- The Raspberry Pi queries the backend and retrieves the correct required PPE list for the worker's role.
- The object detection model runs on the Raspberry Pi 5 and detects at least three PPE classes (helmet, vest, gloves)
- The system grants access (servo opens gate) when all required PPE is detected.
- The system denies access and displays the list of missing items when PPE is incomplete.
- The turnstile display shows a personalized welcome/denial message with the worker's name.
- The Admin Panel allows registration of a worker with an RFID card and a job role.

10.2 Target Success Criteria (Should Have)

- Detection accuracy of $\geq 85\%$ on a test set of PPE images under normal indoor lighting.
- Inspection cycle time (card scan to gate decision) under 10 seconds.
- Role-based PPE requirements correctly differentiated for at least 3 distinct roles (e.g., visitor, technician, construction worker).
- All entry attempts logged with timestamp, identity, detected PPE, and result.
- Admin Panel displays entry log history and compliance statistics.

10.3 Bonus Features (Nice to Have)

- Multi-class detection accuracy benchmark report across all PPE classes.
- Animated instructions on the turnstile display guiding the worker to stand correctly.
- Live camera feed with bounding boxes overlaid on the turnstile display during inspection.
- Admin Panel accessible from multiple devices on the local network simultaneously.
- Configurable PPE requirement profiles for new roles without code changes.

11. References and Resources

AI & Computer Vision:

- Nath, N. D., Behzadan, A. H., & Paal, S. G. (2020). Deep learning for site safety: Real-time detection of personal protective equipment. *Automation in Construction*, 112.
- Ultralytics Construction PPE Dataset: <https://docs.ultralytics.com/datasets/detect/construction-ppe/>
- OpenCV Python Documentation: <https://docs.opencv.org>
- ONNX: <https://onnx.ai>

Hardware:

- Raspberry Pi 5 Documentation: <https://www.raspberrypi.com/documentation>
- Raspberry Pi AI Kit Documentation: <https://www.raspberrypi.com/documentation/accessories/ai-kit.html>
- RC522 RFID Module: <https://components101.com/wireless/rc522-rfid-module>
- Hardware Cost Reference: <https://www.robotistan.com>

Software Libraries:

- REST API Design Guidelines: <https://restfulapi.net>
- Git Version Control: <https://git-scm.com>
- GitHub: <https://github.com>